

IAT 461 / 882 — Assignment 2: Modeling Phigma User Behavior

Name: Marcus Chan

SFU ID: 301455991

Date: July 5 2026

Before you start: read the full assignment README on Canvas.
Run cells top-to-bottom. Do **not** skip cells — later cells depend on earlier ones.
Every cell marked `# YOUR CODE HERE` must be completed.
Every cell marked `**  Your interpretation:**` must be filled in with your own words.

Setup

Run this cell first. It imports all libraries used throughout the notebook.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
import scipy.stats as stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix, classification_report,
    roc_auc_score, roc_curve, ConfusionMatrixDisplay
)
from sklearn.preprocessing import StandardScaler

# Plot style
plt.rcParams.update({
    'figure.dpi': 120,
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.titlesize': 13,
    'axes.labelsize': 11,
```

```

})
ACCENT = '#e8925a'    # course accent colour

print("Libraries loaded.")

```

Libraries loaded.

Part A — Linear Regression: Did the AI Assistant Change User Behavior?

Dataset: phigma_ab_study.csv

Research question: Does access to Phigma's AI design assistant increase whiteboard session duration among free-tier users?

This is a **between-subjects experiment**. Users were randomly assigned to treatment (AI access) or control (no AI). We use OLS regression to estimate the effect of the treatment while controlling for other variables.

No train/test split in this part. We are doing statistical inference — explaining the outcome — not predicting future users.

Load the data

```

In [2]: df_ab = pd.read_csv('phigma_ab_study.csv')

print(f"Shape: {df_ab.shape}")
print(f"\nColumn types:\n{df_ab.dtypes}")
print(f"\nFirst 3 rows:")
df_ab.head(3)

```

Shape: (2000, 12)

Column types:

user_id	str
ab_group	str
device_type	str
account_age_days	int64
prior_whiteboard_sessions	int64
plan_region	str
num_collaborators	int64
feature_tour_completed	int64
templates_used	int64
ai_suggestions_used	int64
whiteboard_actions_per_session	int64
avg_whiteboard_session_duration_min	float64
dtype:	object

First 3 rows:

Out[2]:

	user_id	ab_group	device_type	account_age_days	prior_whiteboard_sessions	plan_regio
0	U76582	control	tablet	478	4	E
1	U49158	control	desktop	633	9	APA
2	U82668	control	desktop	159	10	Na

In [3]: `# Summary statistics`
`df_ab.describe(include='all').round(2)`

Out[3]:

	user_id	ab_group	device_type	account_age_days	prior_whiteboard_sessions	plan_
count	2000	2000	2000	2000.00	2000.00	
unique	1978	2	3	NaN	NaN	
top	U21640	control	desktop	NaN	NaN	
freq	2	1011	1211	NaN	NaN	
mean	NaN	NaN	NaN	356.43	4.51	
std	NaN	NaN	NaN	207.82	3.33	
min	NaN	NaN	NaN	7.00	0.00	
25%	NaN	NaN	NaN	178.00	2.00	
50%	NaN	NaN	NaN	352.50	4.00	
75%	NaN	NaN	NaN	533.25	6.00	
max	NaN	NaN	NaN	729.00	22.00	

In [4]: `# Missing values`
`df_ab.isnull().sum()`

Out[4]:

user_id	0
ab_group	0
device_type	0
account_age_days	0
prior_whiteboard_sessions	0
plan_region	677
num_collaborators	0
feature_tour_completed	0
templates_used	0
ai_suggestions_used	0
whiteboard_actions_per_session	0
avg_whiteboard_session_duration_min	0
dtype:	int64

A1 — Exploratory Data Analysis

A1.1 — Outcome distribution

```
In [5]: # Load target outcome variable
outcome = df_ab['avg_whiteboard_session_duration_min']

# // 1. Normality & Transformation Visualizations
# Initialize three-panel visual layout
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Draw histogram of raw outcome distribution
axes[0].hist(outcome, bins=50, color='skyblue', edgecolor='black')
axes[0].set_title('Raw Outcome Distribution')
axes[0].set_xlabel('Duration (minutes)')
axes[0].set_ylabel('Frequency')

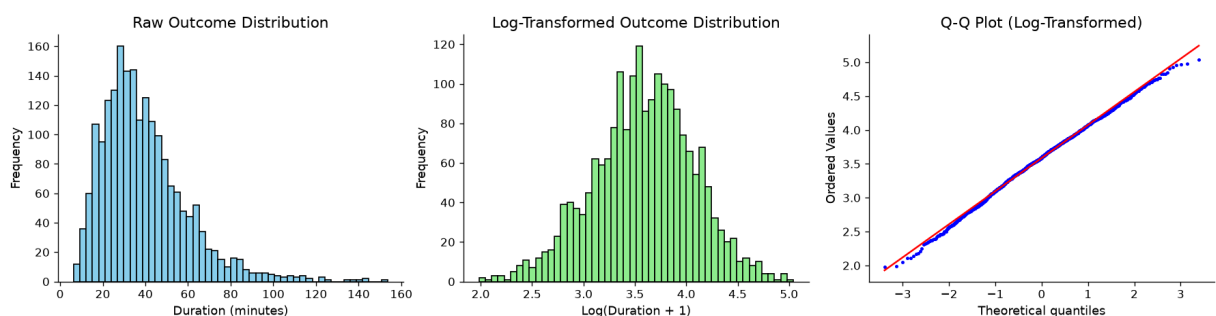
# Transform outcome variable using natural logarithm
log_outcome = np.log(outcome + 1) # avoiding log(0)

# Draw histogram of log-transformed outcome distribution
axes[1].hist(log_outcome, bins=50, color='lightgreen', edgecolor='black')
axes[1].set_title('Log-Transformed Outcome Distribution')
axes[1].set_xlabel('Log(Duration + 1)')
axes[1].set_ylabel('Frequency')

# Draw Q-Q plot comparing log-transformed distribution to a normal distribution
stats.probplot(log_outcome, dist='norm', plot=axes[2])
axes[2].lines[0].set_markersize(2)
axes[2].set_title('Q-Q Plot (Log-Transformed)')

# Render visualizations
plt.tight_layout()
plt.show()

# // 2. Statistical Diagnostics
# Compute and output skewness for both raw and log-transformed distributions
raw_skew = stats.skew(outcome)
log_skew = stats.skew(log_outcome)
print(f"Skewness (raw): {raw_skew:.4f}")
print(f"Skewness (log-transformed): {log_skew:.4f}")
```



Skewness (raw): 1.3758
 Skewness (log-transformed): -0.1627

 **Your interpretation (A1.1):**

Is the log-transformed outcome more normally distributed?

Yes, The log-transformed skewness is much more symmetrical.

How does the Q-Q plot support your conclusion?

The dots match the line meaning the data is following the expected normal distribution after being transformed

Will you use the raw or log outcome in your model?

I will use the log outcome

A1.2 — Examine the confounders

```
In [6]: fig, axes = plt.subplots(1, 2, figsize=(13,5))

#left boxplot of ai suggestions
df_ab.boxplot(column='ai_suggestions_used', by='ab_group', ax=axes[0],
               boxprops=dict(color='darkblue'), medianprops=dict(color='crimson', lw=2),
               whiskerprops=dict(color='darkblue'), capprops=dict(color='black'),
               flierprops=dict(marker='o', color='blue', alpha=0.3, markersize=3)
               )
axes[0].set_xlabel('Group')
axes[0].set_title("AI suggestions used by exp group")
axes[0].set_ylabel('#AI suggestions used')
plt.sca(axes[0])
plt.title("AI suggestions used by group")
plt.suptitle('')

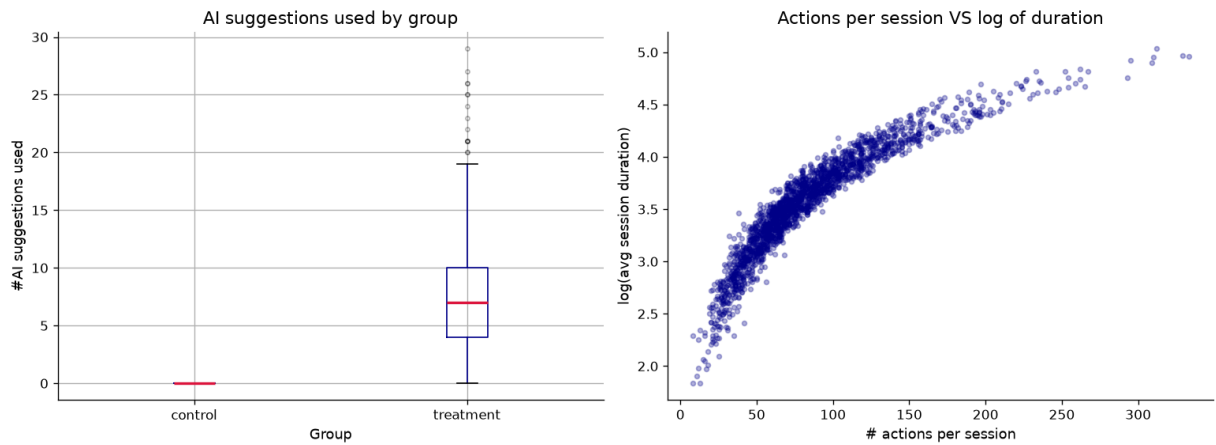
# right side scatter plot of whiteboard actions per session vs log(duration)

log_dur = np.log(df_ab['avg_whiteboard_session_duration_min'])
axes[1].scatter(df_ab['whiteboard_actions_per_session'], log_dur, color='darkblue',
                alpha=0.3, s=12)
axes[1].set_title('Actions per session VS log of duration')
axes[1].set_xlabel('# actions per session')
axes[1].set_ylabel('log(avg session duration)')

plt.tight_layout()
plt.show()

corr_actions = df_ab['whiteboard_actions_per_session'].corr(log_dur)
print(f"Pearson r (whiteboard_actions_per_session vs log_duration): {corr_actions:.2f}")

print("\nDescriptive stats for ai_suggestions_used by group:")
print(df_ab.groupby('ab_group')['ai_suggestions_used'].describe().round(2))
```



Pearson r (whiteboard_actions_per_session vs log_duration): 0.9136

Descriptive stats for ai_suggestions_used by group:

	count	mean	std	min	25%	50%	75%	max
ab_group								
control	1011.0	0.00	0.00	0.0	0.0	0.0	0.0	0.0
treatment	989.0	7.33	4.55	0.0	4.0	7.0	10.0	29.0

Your interpretation (A1.2):

What do you notice about ai_suggestions_used across the two groups?

The control is blank because that group was not given access to AI suggestions

What does the correlation between whiteboard_actions_per_session and the outcome suggest?

Users make more actions as the time passes.

Why should neither be included in your model?

Both do not produce any information worth using as the information is redundant. We want information that will help us show if the AI assistant increases time spent on the whiteboard

A1.3 — Predictor distributions and relationships with outcome

```
In [7]: num_predictors = [
        'account_age_days',
        'prior_whiteboard_sessions',
        'num_collaborators'
      ]

n = len(num_predictors)

fig, axes = plt.subplots(2, n, figsize=(4 * n, 8))
fig.suptitle('Two of the numerical predictors: distributions (top) and vs log(duration) (bottom)',
             fontsize=13, y=1.01)

for j, col in enumerate(num_predictors):
    print("****", col, "****")
```

```

#top row: histogram
axes[0, j].hist(df_ab[col], bins=20, color='darkblue', edgecolor='orange', lw=0)
axes[0, j].set_title(col, fontsize=9)
axes[0, j].set_xlabel(col)
axes[0, j].set_ylabel('Count' if j == 0 else '')

#bottom row: scatterplot
r = df_ab[col].corr(log_dur)
axes[1, j].scatter(df_ab[col], log_dur, color='blue', alpha=0.3, s=10)
axes[1, j].set_xlabel(col)
axes[1, j].set_ylabel('log(duration)' if j == 0 else '')
axes[1, j].annotate(f'r = {r:.2f}', xy=(0.05, 0.9),
                    xycoords='axes fraction', fontsize=8, color='crimson')

plt.tight_layout()
plt.show()

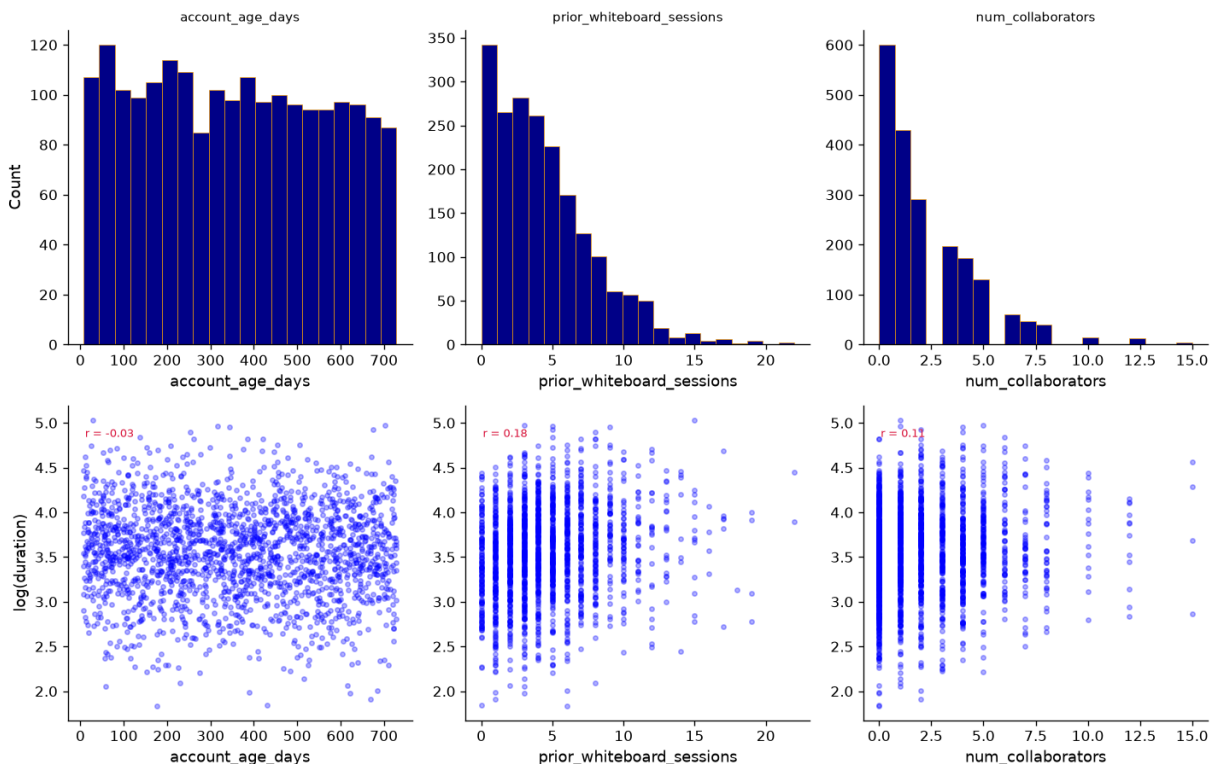
```

**** account_age_days ****

**** prior_whiteboard_sessions ****

**** num_collaborators ****

Two of the numerical predictors: distributions (top) and vs log(duration) (bottom)



```

In [8]: cat_predictors = ['ab_group', 'device_type', 'plan_region']

log_dur = np.log(df_ab['avg_whiteboard_session_duration_min'])

fig, axes = plt.subplots(1, len(cat_predictors), figsize=(13,5))
fig.suptitle('log(session duration) by categorical predictors', fontsize=13)

for j, col in enumerate(cat_predictors):
    levels = sorted(df_ab[col].dropna().unique())
    grouped = [log_dur[df_ab[col] == lvl].values for lvl in levels]
    axes[j].boxplot(grouped, tick_labels=levels, patch_artist=True,

```

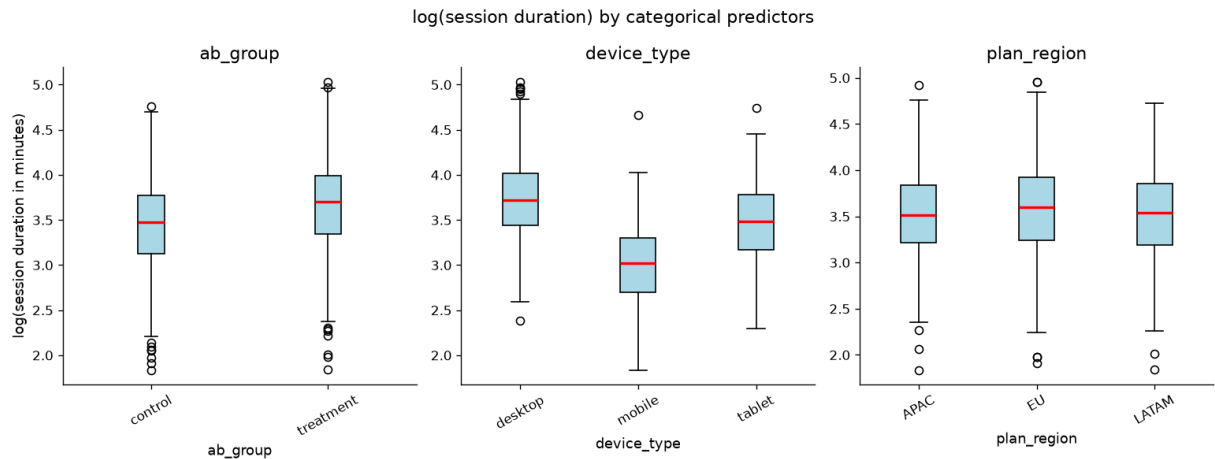
```

boxprops=dict(facecolor="lightblue"), medianprops=dict(color='red')

axes[j].set_title(col)
axes[j].set_xlabel(col)
axes[j].set_ylabel('log(session duration in minutes)' if j == 0 else '')
axes[j].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.show()

```



Your interpretation (A1.3):

3–5 sentences summarizing your EDA. Which predictors look most promising? Are any right-skewed and worth log-transforming before use as predictors? Which two features are you excluding and why?

The EDA reveals that `prior_whiteboard_sessions` and `num_collaborators` are both right-skewed and show the strongest positive correlations with $\log(\text{duration})$ ($r = 0.18$ and $r = 0.11$), making them the most promising numerical predictors.

`account_age_days` is roughly uniform with near-zero correlation ($r = -0.03$), therefore not contributing much. Among categorical predictors, `device_type` stands out most as the mobile users have a much lower median than desktop and tablet users. The `ab_group` and `plan_region` boxplots show the medians only differ a bit so they don't look too promising.

Two features are excluded:

- `whiteboard_actions_per_session` is caused by the treatment itself, so including it would soak up part of the treatment effect making it look less effective.
- `ai_suggestions_used` is excluded because control-group users always have a value of zero.

A2 — Feature Engineering

Prepare your feature matrix for OLS. Exclude the two problematic variables identified in A1.

```
In [9]: df_ab['ab_group_enc'] = (df_ab['ab_group'] == 'treatment').astype(int)
device_dummies = pd.get_dummies(df_ab['device_type'], prefix='device', drop_first=True)
region_dummies = pd.get_dummies(df_ab['plan_region'], prefix='region', drop_first=True)

log_account_age = np.log1p(df_ab['account_age_days'])
log_prior_whiteboard_sessions = np.log1p(df_ab['prior_whiteboard_sessions'])
log_num_collaborators = np.log1p(df_ab['num_collaborators'])

X = pd.concat([
    df_ab['ab_group_enc'].rename('ab_group'),
    log_account_age.rename('log_account_age_days'),
    log_prior_whiteboard_sessions.rename('log_whiteboard_sessions'),
    log_num_collaborators.rename('log_num_collaborators'),
    device_dummies,
    region_dummies
], axis=1)

y = np.log(df_ab['avg_whiteboard_session_duration_min'])

assert X.isnull().sum().sum() == 0
print(f"Feature matrix shape: {X.shape}")
print(f"Missing values in X: {X.isnull().sum().sum()}")
print(f"\nFeature columns:\n{X.columns.tolist()}")
X.head(3)
```

Feature matrix shape: (2000, 8)

Missing values in X: 0

Feature columns:

['ab_group', 'log_account_age_days', 'log_whiteboard_sessions', 'log_num_collaborators', 'device_mobile', 'device_tablet', 'region_EU', 'region_LATAM']

```
Out[9]:
```

	ab_group	log_account_age_days	log_whiteboard_sessions	log_num_collaborators	device
0	0	6.171701	1.609438	1.386294	
1	0	6.452049	2.302585	0.693147	
2	0	5.075174	2.397895	1.791759	

Your interpretation (A2):

List your final features. For each one, write one sentence justifying its inclusion and any transformation applied.

Your interpretation (A2):

- **ab_group** : The primary variable of interest; encodes whether the user received the new onboarding flow (1) or the original (0).
- **log_account_age_days** : Included to control for user experience level; log-transformed because account age was applied consistently with the other count

variables.

- **log_whiteboard_sessions** : Prior usage is a strong predictor of engagement; log-transformed because the distribution is right-skewed with a long tail.
- **log_num_collaborators** : Users who collaborate more tend to have longer sessions; log-transformed because the distribution is heavily right-skewed.
- **device_mobile** : Mobile users showed substantially shorter sessions in EDA, so device type must be controlled for; desktop is the reference category.
- **device_tablet** : Tablet users also showed lower session duration than desktop users, captured as a separate dummy from mobile.
- **region_EU** : Captures regional differences
- **region_LATAM** : Captures regional differences,

A3 — OLS Regression

```
In [10]: # YOUR CODE HERE
# Add a constant for the intercept, then fit OLS
X_const = sm.add_constant(X)
model_ols = sm.OLS(y, X_const).fit(cov_type='HC3')
print(model_ols.summary())
```

OLS Regression Results

```

=====
=====
Dep. Variable:      avg_whiteboard_session_duration_min    R-squared:
0.354
Model:              OLS    Adj. R-squared:
0.351
Method:              Least Squares    F-statistic:
122.2
Date:                Sun, 05 Jul 2026    Prob (F-statistic):
1.17e-166
Time:                18:40:33    Log-Likelihood:
-1034.8
No. Observations:    2000    AIC:
2088.
Df Residuals:        1991    BIC:
2138.
Df Model:            8
Covariance Type:     HC3
=====
=====

```

	coef	std err	z	P> z	[0.025
0.975]					

const	3.4148	0.063	54.329	0.000	3.292
3.538					
ab_group	0.2184	0.018	11.961	0.000	0.183
0.254					
log_account_age_days	-0.0170	0.010	-1.697	0.090	-0.037
0.003					
log_whiteboard_sessions	0.1414	0.015	9.703	0.000	0.113
0.170					
log_num_collaborators	0.0878	0.013	6.817	0.000	0.063
0.113					
device_mobile	-0.7244	0.028	-26.017	0.000	-0.779
-0.670					
device_tablet	-0.2565	0.022	-11.890	0.000	-0.299
-0.214					
region_EU	0.0296	0.021	1.435	0.151	-0.011
0.070					
region_LATAM	-0.0563	0.033	-1.713	0.087	-0.121
0.008					
=====					
Omnibus:	0.913	Durbin-Watson:		2.046	
Prob(Omnibus):	0.633	Jarque-Bera (JB):		0.970	
Skew:	0.026	Prob(JB):		0.616	
Kurtosis:	2.906	Cond. No.		43.1	
=====					

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

A3.1 — Interpret the model

```

In [11]: print(f"R²                = {model_ols.rsquared:.4f}")
        print(f"Adj. R²          = {model_ols.rsquared_adj:.4f}")
        print()

        ab_coef = model_ols.params['ab_group']
        ab_pval = model_ols.pvalues['ab_group']

        #y = Log(duration)

        ab_pct_change = (np.exp(ab_coef) - 1)

        control_mean = df_ab.loc[df_ab['ab_group'] == 'control', 'avg_whiteboard_session_du

        ab_mins_effect = control_mean * (np.exp(ab_coef) - 1)

        print(f"ab_group coefficient : {ab_coef:.4f}")
        print(f"p-value              : {ab_pval:.4f}")
        print(f"Back-transformed      : {ab_pct_change:.1%} relative to control mean")
        print(f"                    : ~= {ab_mins_effect:.1f} min above control mean ({co

        # significant predictors with p < 0.05
        sig = model_ols.pvalues[model_ols.pvalues < 0.05].sort_values()
        print('significant predictors are: ')
        for var, pv in sig.items():
            coef = model_ols.params[var]
            print(f" {var:<35s}  coef = {coef:.4f}  p={pv:.4f}")

```

R² = 0.3538
Adj. R² = 0.3512

ab_group coefficient : 0.2184
p-value : 0.0000
Back-transformed : 24.4% relative to control mean
: ~= 8.6 min above control mean (35.2 min)

significant predictors are:

const	coef = 3.4148	p=0.0000
device_mobile	coef = -0.7244	p=0.0000
ab_group	coef = 0.2184	p=0.0000
device_tablet	coef = -0.2565	p=0.0000
log_whiteboard_sessions	coef = 0.1414	p=0.0000
log_num_collaborators	coef = 0.0878	p=0.0000

Results paragraph (A3):

Write a results paragraph as if reporting in a research paper. Include:

- **The treatment effect estimate and its significance**
- **R² and what it tells you about model fit**
- **At least two other notable predictors and their direction**
- **Whether the AI assistant's effect is practically meaningful (not just statistically significant)**

The OLS model estimated a treatment effect of $\beta = 0.2184$ on $\log(\text{session duration})$ ($p < 0.001$), indicating that access to the AI assistant is associated with an approximate 24.4%

increase in average whiteboard session duration, equivalent to roughly 8.6 additional minutes per session relative to the control-group mean of 35.2 minutes. This effect was statistically significant well beyond the 0.05 threshold. The model explained $R^2 = 0.354$ of the variance in $\log(\text{session duration})$, suggesting that treatment assignment and the included covariates account for a substantial, though far from complete, share of the variation in engagement, with the remaining ~64.6% attributable to unmeasured factors such as individual user motivation or context of use. Among the other significant predictors, device type had the largest effect: mobile users had substantially shorter sessions than desktop users ($\beta = -0.7244$, $p < 0.001$), a magnitude more than three times larger than the treatment effect, suggesting that device constraints such as screen size and input method shape engagement more than any single feature. Prior whiteboard usage was also positively associated with session length ($\beta = 0.1414$, $p < 0.001$), consistent with more experienced users engaging more deeply per session. Practically, a 24.4% increase representing nearly 9 extra minutes per session is a meaningful improvement, not merely a statistically detectable one: given the large sample size ($n = 2000$), even trivial effects would reach significance, but an effect of this magnitude reflects a genuine shift in user behavior rather than a negligible artifact of sample size.

A3.2 — Regression diagnostics

```
In [12]: # Extract predictions and error terms (residuals) from regression model
fitted_vals = model_ols.fittedvalues
residuals = model_ols.resid

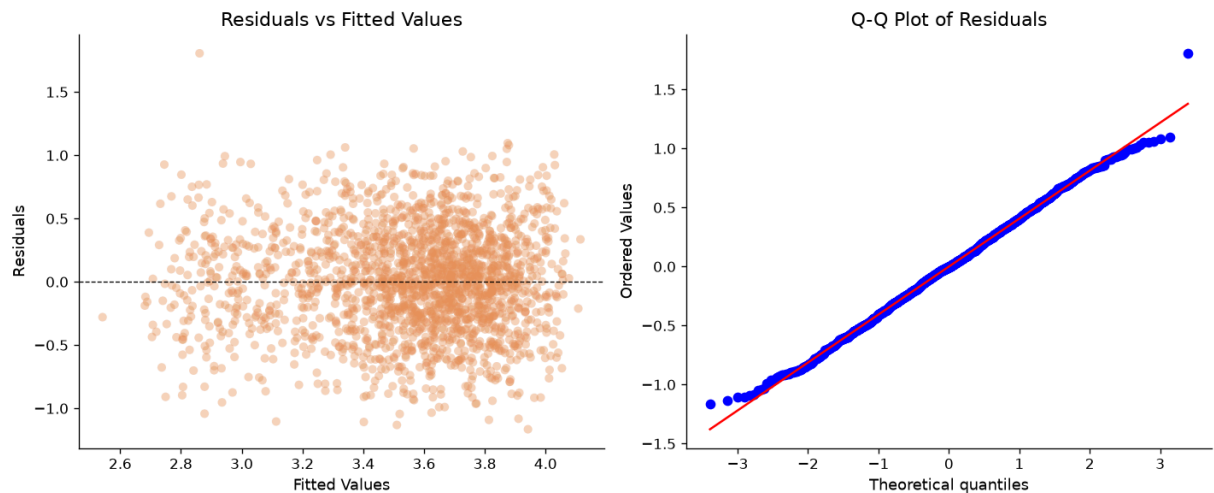
# Initialize a side-by-side diagnostic visualization layout
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# // Check for uniform variance (homoscedasticity)
# Draw scatter plot of prediction values vs residuals
axes[0].scatter(fitted_vals, residuals, alpha=0.4, color=ACCENT, edgecolor='none')

# Overlay a horizontal zero-error baseline reference line
axes[0].axhline(0, color='black', linestyle='--', linewidth=0.8)
axes[0].set_title('Residuals vs Fitted Values')
axes[0].set_xlabel('Fitted Values')
axes[0].set_ylabel('Residuals')

# // Check for normality of errors
# Draw a Q-Q plot of residuals against a normal theoretical distribution
stats.probplot(residuals, dist='norm', plot=axes[1])
axes[1].set_title('Q-Q Plot of Residuals')

# Render diagnostic plots
plt.tight_layout()
plt.show()
```



Your interpretation (A3.2):

Do the residuals look randomly scattered around zero?

Yes, the points are spread evenly above and below the zero line with no funnel shape, so the model's variance looks consistent across fitted values.

Does the Q-Q plot suggest normally distributed errors?

Mostly. The points follow the diagonal line closely in the middle, but there's a slight deviation at the tails, especially one point in the upper right that sits above the line.

Note any patterns that concern you.

Just one extreme outlier shows up in both plots. Other than that, there's no clear pattern or curve that would suggest a problem with the model.

A4 — Exploratory Interaction Analysis

The average treatment effect hides heterogeneity — the AI assistant may help some users more than others.

State your hypothesis here (before writing any code):

Which interaction are you testing? Which two variables? What is your theoretical reason for expecting this interaction? Which group do you predict will benefit more from the AI assistant?

Interaction tested: `ab_group` × `log_prior_whiteboard_sessions`

Theoretical reason: The AI assistant provides suggestions and guidance that are most useful when a user is unfamiliar with the tool. Users with more prior sessions are already comfortable completing tasks on their own and are less likely to rely on or benefit from AI suggestions. As a result, the treatment effect should be larger for newer users and smaller for veterans who have little need for additional assistance.

Prediction: Users with fewer prior whiteboard sessions will benefit more from the AI assistant, showing a larger increase in session duration under treatment. The interaction coefficient on `ab_group × log_prior_whiteboard_sessions` should be negative: as prior experience increases, the additional effect of the treatment diminishes.

```
In [13]: # Step 1: Create the interaction term
X_int = X.copy()
X_int['ab_x_log_whiteboard'] = X_int['ab_group'] * X_int['log_whiteboard_sessions']

# Step 2: Add constant and refit OLS
model_int = sm.OLS(y, sm.add_constant(X_int)).fit(cov_type='HC3')

print(f"R²          = {model_int.rsquared:.4f}")
print(f"Adj R²       = {model_int.rsquared_adj:.4f}\n")
for p in ['ab_group', 'log_whiteboard_sessions', 'ab_x_log_whiteboard']:
    print(f"{p:<35s} coef = {model_int.params[p]:+.4f} p = {model_int.pvalues[p]:
```

R² = 0.3548

Adj R² = 0.3518

ab_group	coef = +0.1446	p = 0.0024
log_whiteboard_sessions	coef = +0.1174	p = 0.0000
ab_x_log_whiteboard	coef = +0.0487	p = 0.0964

```
In [14]: # YOUR CODE HERE
# Visualize the interaction
# Example: plot mean log(outcome) for treatment vs control, split by the moderating
# Use a grouped bar chart or overlapping regression lines

ws_range = np.linspace(X_int['log_whiteboard_sessions'].min(),
                        X_int['log_whiteboard_sessions'].max(), 200)

row_means = X_int.mean().values

def pred_line(ab_val):
    df = pd.DataFrame(np.tile(row_means, (len(ws_range), 1)), columns=X_int.columns)
    df['ab_group'] = ab_val
    df['log_whiteboard_sessions'] = ws_range
    df['ab_x_log_whiteboard'] = ab_val * ws_range
    df.insert(0, 'const', 1.0)
    return model_int.predict(sm.add_constant(df))

pred_ctrl = pred_line(0)
pred_trt = pred_line(1)

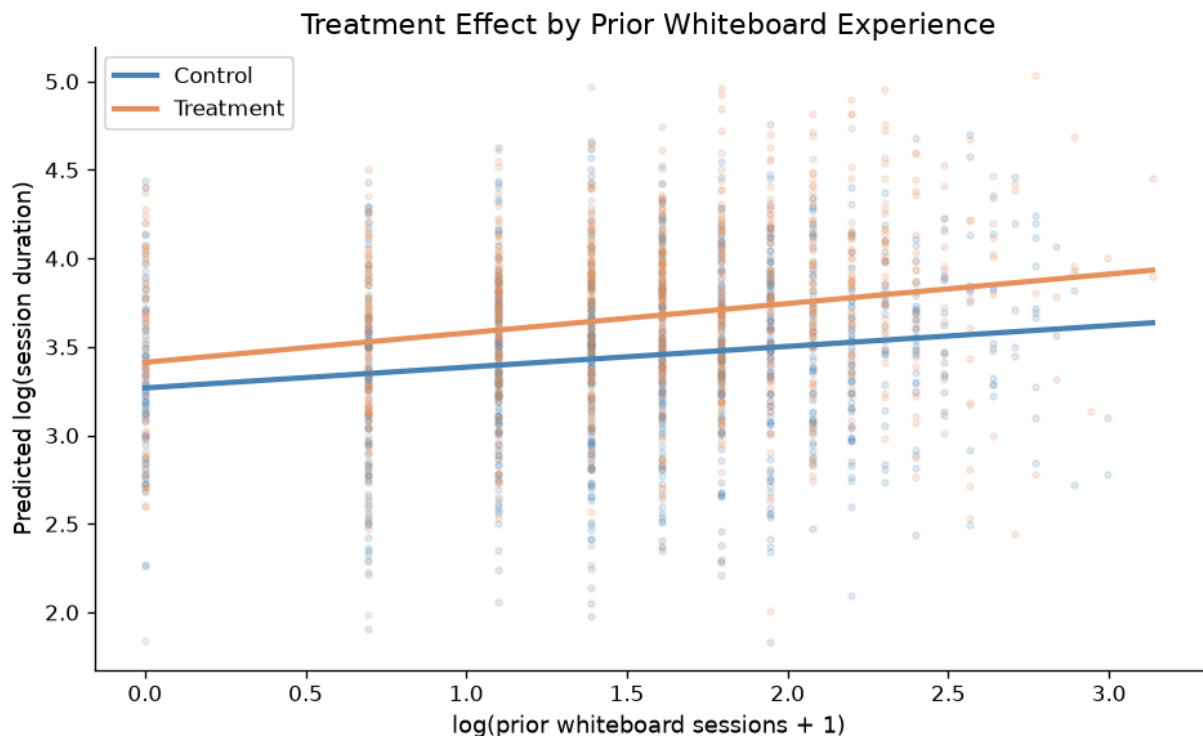
fig, ax = plt.subplots(figsize=(8, 5))
ctrl_mask = (df['ab_group'] == 'control').values
trt_mask = (df['ab_group'] == 'treatment').values
ax.scatter(X_int.loc[ctrl_mask, 'log_whiteboard_sessions'], y[ctrl_mask],
           alpha=0.15, s=8, color='steelblue')
ax.scatter(X_int.loc[trt_mask, 'log_whiteboard_sessions'], y[trt_mask],
           alpha=0.15, s=8, color=ACCENT)
ax.plot(ws_range, pred_ctrl, color='steelblue', lw=2.5, label='Control')
ax.plot(ws_range, pred_trt, color=ACCENT, lw=2.5, label='Treatment')
ax.set_xlabel('log(prior whiteboard sessions + 1)')
```

```

ax.set_ylabel('Predicted log(session duration)')
ax.set_title('Treatment Effect by Prior Whiteboard Experience')
ax.legend()
plt.tight_layout()
plt.show()

print("Interaction term coefficient:", model_int.params['ab_x_log_whiteboard'])
print("Interaction term p-value:", model_int.pvalues['ab_x_log_whiteboard'])

```



Interaction term coefficient: 0.048733488501939454

Interaction term p-value: 0.09641008449548989

Your interpretation (A4):

Was your hypothesis supported? What does the interaction coefficient tell you about which users benefit most from the AI assistant? What does this imply for how Phigma should roll out the feature?

The hypothesis was not supported. The interaction coefficient on `ab_group × log_prior_whiteboard_sessions` was $\beta = 0.049$. This tells us that the treatment effect grows very slightly as prior experience increases, but the result was not statistically significant ($p = 0.096$), so this pattern could be due to chance.

For Phigma, this means there is no strong basis for targeting the AI assistant rollout at a specific experience segment. A uniform rollout is reasonable, though the trend in the data (experienced users responding slightly more) would be worth revisiting with a larger sample if Phigma wants to make experience-based targeting decisions.

Part B — Logistic Regression: Who Converts?

Dataset: phigma_subscription.csv

Research question: Which behavioral and demographic factors predict whether a free-tier user subscribes within 30 days?

This dataset requires **feature engineering before modeling**. Several columns arrive as raw datetimes or aggregate counts that need transformation.

```
In [15]: df_sub = pd.read_csv('phigma_subscription.csv')

print(f"Shape: {df_sub.shape}")
print(f"\nColumn types:\n{df_sub.dtypes}")
df_sub.head(3)
```

Shape: (3000, 15)

Column types:

```
user_id                str
signup_date            str
first_session_date     str
device_type            str
country_tier           str
referral_source        str
num_sessions_30d       int64
total_time_min_30d     int64
projects_created_30d   int64
exports_30d            int64
collaborators_invited_30d int64
support_tickets_30d    int64
used_premium_feature_preview int64
ai_assistant_sessions_30d int64
subscribed             int64
dtype: object
```

```
Out[15]:
```

	user_id	signup_date	first_session_date	device_type	country_tier	referral_source	num_
0	V94735	2024-05-23	2024-05-28	tablet	tier_1	organic	
1	V15501	2024-03-05	2024-03-05	tablet	tier_2	social	
2	V98542	2024-01-05	2024-01-19	desktop	tier_1	organic	

```
In [16]: df_sub.describe(include='all').round(2)
```

Out[16]:

	user_id	signup_date	first_session_date	device_type	country_tier	referral_source
count	3000	3000	3000	3000	3000	3000
unique	2947	181	200	3	3	4
top	V43688	2024-06-06	2024-06-27	desktop	tier_1	organic
freq	3	33	29	1788	1332	1153
mean	NaN	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN	NaN

B1 — Feature Engineering

Derive the required features listed in the README before doing any modeling or EDA.

In [17]:

```
# YOUR CODE HERE

# 1. Parse dates
df_sub['signup_date'] = pd.to_datetime(df_sub['signup_date'])
df_sub['first_session_date'] = pd.to_datetime(df_sub['first_session_date'])

# 2. days_to_first_session
df_sub['days_to_first_session'] = (df_sub['first_session_date'] - df_sub['signup_date']).dt.days

# 3. avg_session_duration_min - handle divide-by-zero explicitly
df_sub['avg_session_duration_min'] = np.where(
    df_sub['num_sessions_30d'] == 0,
    0,
    df_sub['total_time_min_30d'] / df_sub['num_sessions_30d']
)

# 4. is_weekend_signup (1 if Saturday=5 or Sunday=6)
df_sub['is_weekend_signup'] = df_sub['signup_date'].dt.weekday.isin([5, 6]).astype(int)

# 5. One-hot encode device_type (drop first level)
device_dummies2 = pd.get_dummies(df_sub['device_type'], prefix='device', drop_first=True)

# 6. One-hot encode referral_source (drop first level)
referral_dummies = pd.get_dummies(df_sub['referral_source'], prefix='ref', drop_first=True)

# 7. Encode country_tier - ordinal or one-hot (justify below)
```

```

df_sub['country_tier_enc'] = df_sub['country_tier'].map({'tier_1': 1, 'tier_2': 2,

# 8. Assemble final feature matrix
drop_cols = ['user_id', 'signup_date', 'first_session_date',
              'device_type', 'referral_source', 'country_tier', 'subscribed']
X_sub = pd.concat([
    df_sub.drop(columns=drop_cols),
    device_dummies2,
    referral_dummies
], axis=1)
y_sub = df_sub['subscribed']

# Confirm
print(f"Any inf: {np.isinf(X_sub.astype(float).values).any()}")
print(f"Any NaN: {X_sub.isnull().any().any()}")
print(f"Feature matrix shape: {X_sub.shape}")
X_sub.head(3)

```

Any inf: False

Any NaN: False

Feature matrix shape: (3000, 17)

Out[17]:

	num_sessions_30d	total_time_min_30d	projects_created_30d	exports_30d	collaborators_
0	1	96	1	0	
1	15	267	8	1	
2	16	263	0	0	

Your interpretation (B1):

For each engineered feature, one sentence: why might it predict subscription? Also describe how you handled the divide-by-zero edge case for `avg_session_duration_min`.

- `days_to_first_session` : Shows how long a user takes to subscribe. Can be as a baseline to determine if certain features are more likely to decrease this time.
- `avg_session_duration_min` : Longer average sessions suggest deeper engagement, possibly making subscription more likely.
- `is_weekend_signup` : Users who sign up on weekends may be exploring tools for personal or freelance use, suggesting personal interest rather than work obligation.
- `device_type` : How users are using the product.
- `referral_source` : Which method of marketing is the most successful
- For `avg_session_duration_min`, a divide-by-zero edge case arises when `num_sessions_30d` is 0. The value was set to 0 using `np.where`, on the assumption that a user with no sessions has an average duration of zero.

B2 — EDA for Subscription

```
In [18]: # Subscription rate
rate = df_sub['subscribed'].mean()
n_pos = df_sub['subscribed'].sum()
n_total = len(df_sub)
print(f"Subscription rate: {rate:.1%} ({n_pos} / {n_total})")
print(f"Class balance - 0: {n_total - n_pos} | 1: {n_pos}")
```

Subscription rate: 20.8% (625 / 3000)

Class balance - 0: 2375 | 1: 625

```
In [19]: # YOUR CODE HERE
# Visualize subscription rates across at least 3 features
# Suggestions:
# - Bar chart: subscription rate by referral_source
# - Bar chart: subscription rate by country_tier
# - Boxplot: exports_30d distribution by subscribed (0 vs 1)
# - Bar chart: subscription rate by used_premium_feature_preview
# - Any other feature you find interesting

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

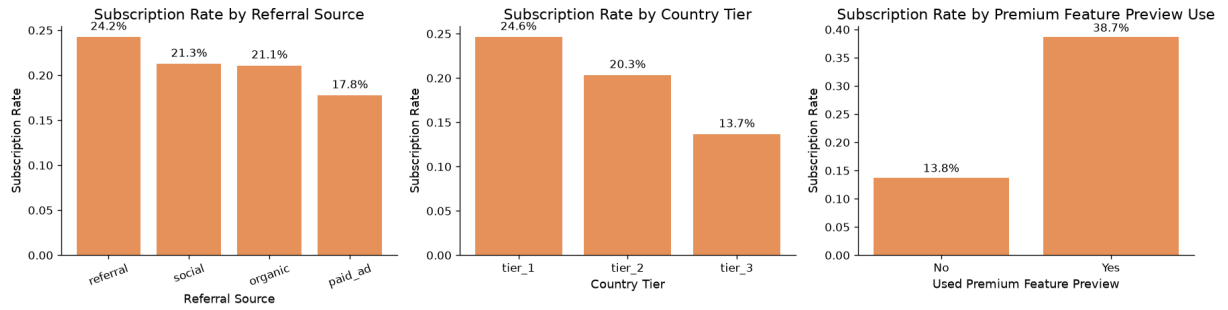
# Plot 1 - subscription rate by referral_source
sub_by_ref = df_sub.groupby('referral_source')['subscribed'].mean().sort_values(ascending=True)
axes[0].bar(sub_by_ref.index, sub_by_ref.values, color=ACCENT)
axes[0].set_title('Subscription Rate by Referral Source')
axes[0].set_xlabel('Referral Source')
axes[0].set_ylabel('Subscription Rate')
axes[0].tick_params(axis='x', rotation=20)

# Plot 2 - subscription rate by country_tier
sub_by_tier = df_sub.groupby('country_tier')['subscribed'].mean().sort_index(ascending=True)
axes[1].bar(sub_by_tier.index, sub_by_tier.values, color=ACCENT)
axes[1].set_title('Subscription Rate by Country Tier')
axes[1].set_xlabel('Country Tier')
axes[1].set_ylabel('Subscription Rate')

# Plot 3 - subscription rate by used_premium_feature_preview
sub_by_premium = df_sub.groupby('used_premium_feature_preview')['subscribed'].mean()
labels = ['No' if k == 0 else 'Yes' for k in sub_by_premium.index]
axes[2].bar(labels, sub_by_premium.values, color=ACCENT)
axes[2].set_title('Subscription Rate by Premium Feature Preview Use')
axes[2].set_xlabel('Used Premium Feature Preview')
axes[2].set_ylabel('Subscription Rate')

for ax in axes:
    for bar in ax.patches:
        ax.text(bar.get_x() + bar.get_width() / 2,
                bar.get_height() + 0.005,
                f'{bar.get_height():.1%}',
                ha='center', va='bottom', fontsize=10)

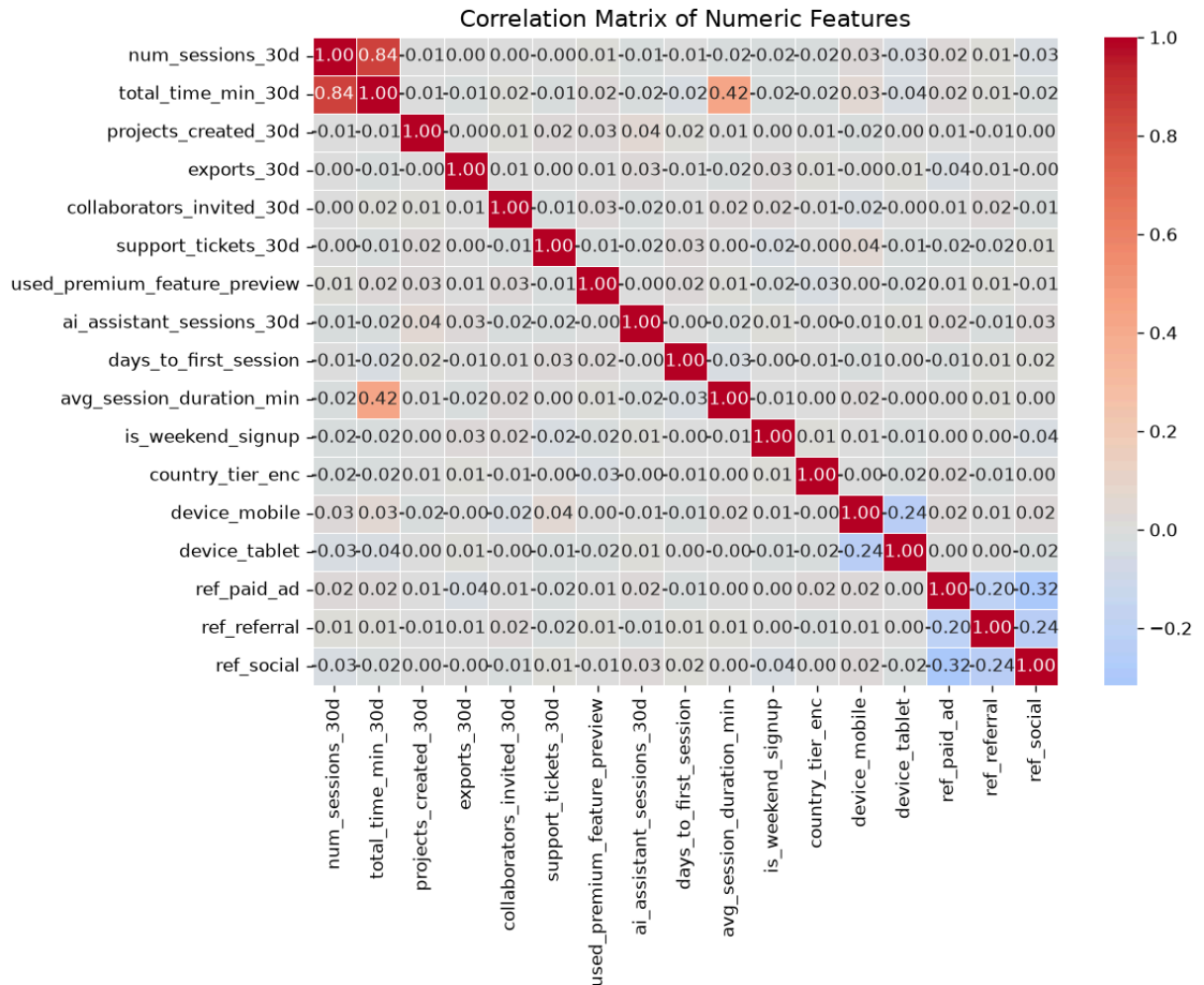
plt.tight_layout()
plt.show()
```



```
In [20]: # YOUR CODE HERE
# Correlation matrix of numeric features (exclude user_id and binary outcome)
# numeric_features = X_sub.select_dtypes(include='number').columns.tolist()
# corr = X_sub[numeric_features].corr()
# Plot as heatmap

corr = X_sub.astype(float).corr()

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0,
            linewidths=0.5, ax=ax)
ax.set_title('Correlation Matrix of Numeric Features')
plt.tight_layout()
plt.show()
# END
```



Your interpretation (B2):

Which two or three features appear most strongly associated with subscription in your EDA? Are any findings counterintuitive? Note any features that appear highly correlated with each other — does this concern you?

The strongest predictor visible in the EDA is `used_premium_feature_preview`: users who tried a premium feature converted at roughly 39% compared to only 14% for those who did not. This suggests that there is a moderately high likelihood that preview correlated positively to users subscribing.

`country_tier` also shows a clear monotonic pattern, with tier_1 users subscribing at ~25%, dropping to ~20% for tier_2 and ~14% for tier_3, consistent with higher purchasing power in wealthier markets.

`referral_source` showed that a referral had about a 24% subscription rate with social and organic means following closely at around 21%. Paid ads performed a bit worse at around 18%.

None of these findings are counterintuitive. The premium preview result could be from user's who are already inclined on subscribing, however this would require further analysis.

The most concerning correlation in the feature matrix is between `num_sessions_30d` and `total_time_min_30d` ($r = 0.84$), as these two features are highly redundant. This may inflate standard errors on both coefficients, making them individually harder to interpret. The negative correlations among the referral and device dummies are expected since these are mutually exclusive categories.

B3 — Logistic Regression with statsmodels (Inferential)

We fit on the **full dataset** here to maximize statistical power for inference.

No train/test split — the goal is to interpret coefficients, not evaluate prediction.

```
In [21]: # YOUR CODE HERE
X_const_sub = sm.add_constant(X_sub.astype(float))
model_logit = sm.Logit(y_sub, X_const_sub).fit()
print(model_logit.summary())
```

Optimization terminated successfully.
 Current function value: 0.440520
 Iterations 6

Logit Regression Results

```

=====
Dep. Variable:          subscribed    No. Observations:          3000
Model:                  Logit         Df Residuals:              2982
Method:                 MLE          Df Model:                  17
Date:                  Sun, 05 Jul 2026    Pseudo R-squ.:           0.1392
Time:                  18:40:34          Log-Likelihood:          -1321.6
converged:              True            LL-Null:                 -1535.2
Covariance Type:        nonrobust        LLR p-value:             3.542e-80
=====
=====
                                coef      std err          z      P>|z|      [0.025
0.975]
-----
const                    -2.5253      0.325      -7.772      0.000      -3.162
-1.889
num_sessions_30d         0.0425      0.028       1.544      0.123      -0.011
0.097
total_time_min_30d      -0.0010      0.001      -0.948      0.343      -0.003
0.001
projects_created_30d    -0.0098      0.021      -0.456      0.648      -0.052
0.032
exports_30d              0.1941      0.022       8.873      0.000       0.151
0.237
collaborators_invited_30d 0.0613      0.022       2.755      0.006       0.018
0.105
support_tickets_30d      0.1547      0.043       3.588      0.000       0.070
0.239
used_premium_feature_preview 1.4695      0.100     14.720      0.000       1.274
1.665
ai_assistant_sessions_30d 0.0200      0.017       1.197      0.231      -0.013
0.053
days_to_first_session   -0.0927      0.014      -6.737      0.000      -0.120
-0.066
avg_session_duration_min  0.0311      0.010       3.050      0.002       0.011
0.051
is_weekend_signup        0.0482      0.107       0.448      0.654      -0.163
0.259
country_tier_enc         -0.3566      0.065      -5.449      0.000      -0.485
-0.228
device_mobile            -0.1454      0.143      -1.014      0.310      -0.426
0.136
device_tablet            -0.2168      0.118      -1.838      0.066      -0.448
0.014
ref_paid_ad              -0.1825      0.136      -1.341      0.180      -0.449
0.084
ref_referral             0.2143      0.150       1.425      0.154      -0.080
0.509
ref_social               0.0555      0.122       0.454      0.650      -0.184
0.295
=====
=====

```

```
In [22]: # YOUR CODE HERE
# Compute odds ratios for all significant predictors (p < 0.05)
odds_ratios = np.exp(model_logit.params)
conf_int     = np.exp(model_logit.conf_int())

results = pd.DataFrame({
    'odds_ratio': odds_ratios,
    'ci_lower':   conf_int[0],
    'ci_upper':   conf_int[1],
    'p_value':    model_logit.pvalues
})

sig = results[results['p_value'] < 0.05].sort_values('odds_ratio', ascending=False)
print(sig.round(4))
```

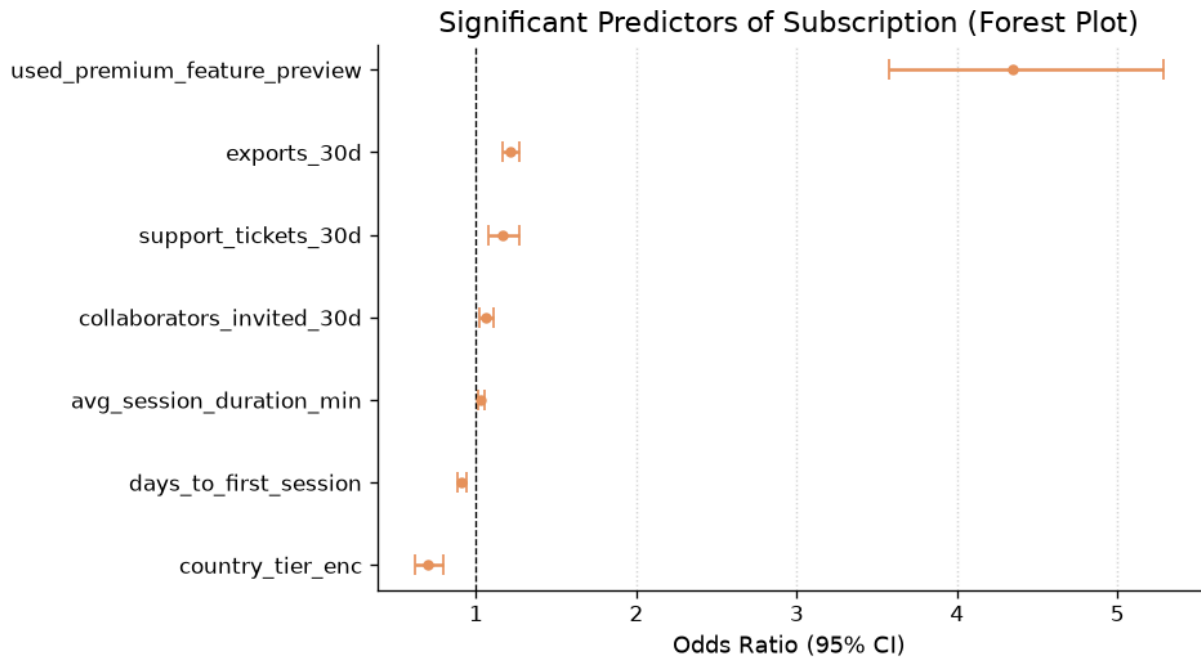
	odds_ratio	ci_lower	ci_upper	p_value
used_premium_feature_preview	4.3470	3.5745	5.2864	0.0000
exports_30d	1.2142	1.1633	1.2674	0.0000
support_tickets_30d	1.1673	1.0727	1.2702	0.0003
collaborators_invited_30d	1.0632	1.0178	1.1106	0.0059
avg_session_duration_min	1.0315	1.0112	1.0523	0.0023
days_to_first_session	0.9114	0.8872	0.9364	0.0000
country_tier_enc	0.7001	0.6158	0.7959	0.0000
const	0.0800	0.0423	0.1513	0.0000

```
In [23]: # YOUR CODE HERE
# Visualize odds ratios as a horizontal bar chart (forest plot style)
# Show confidence intervals if possible
# Order bars by magnitude

sig_sorted = sig.drop('const', errors='ignore').sort_values('odds_ratio')

fig, ax = plt.subplots(figsize=(8, len(sig_sorted) * 0.5 + 1))

ax.errorbar(sig_sorted['odds_ratio'], sig_sorted.index,
            xerr=[sig_sorted['odds_ratio'] - sig_sorted['ci_lower'],
                  sig_sorted['ci_upper'] - sig_sorted['odds_ratio']],
            fmt='o', color=ACCENT, capsize=5, linewidth=1.5, markersize=4)
ax.axvline(1, color='black', linestyle='--', linewidth=0.8)
ax.set_xlabel('Odds Ratio (95% CI)')
ax.set_title('Significant Predictors of Subscription (Forest Plot)')
ax.grid(axis='x', linestyle=':', alpha=0.5)
plt.tight_layout()
plt.show()
```

Results paragraph (B3):

Interpret at least three odds ratios in plain English (e.g. "Users who previewed a premium feature were X times more likely to subscribe..."). Identify at least one predictor whose sign or magnitude surprised you and offer an explanation.

Users who previewed a premium feature were approximately 4.3 times more likely to subscribe.

Each export was associated with slightly higher subscription odds 1.2 times, suggesting that users actively using the Phigma slightly more likely to subscribe.

Higher `country_tier_enc` values were associated with lower subscription odds at 0.7 times.

Surprisingly, `support_tickets_30d` has a positive odds ratio (or about 1.1 times). Rather than indicating frustration, submitting tickets may signal users who are engaged enough to troubleshoot rather than abandon the product.

B4 — Logistic Regression with sklearn (Predictive)

```
In [24]: # YOUR CODE HERE
#Train/test split – use these exact parameters so Part C comparisons are valid
X_train, X_test, y_train, y_test = train_test_split(
    X_sub, y_sub, test_size=0.2, random_state=42, stratify=y_sub
)
print(f"Train: {X_train.shape} | Test: {X_test.shape}")
print(f"Train positive rate: {y_train.mean():.3f} | Test positive rate: {y_test.m
```

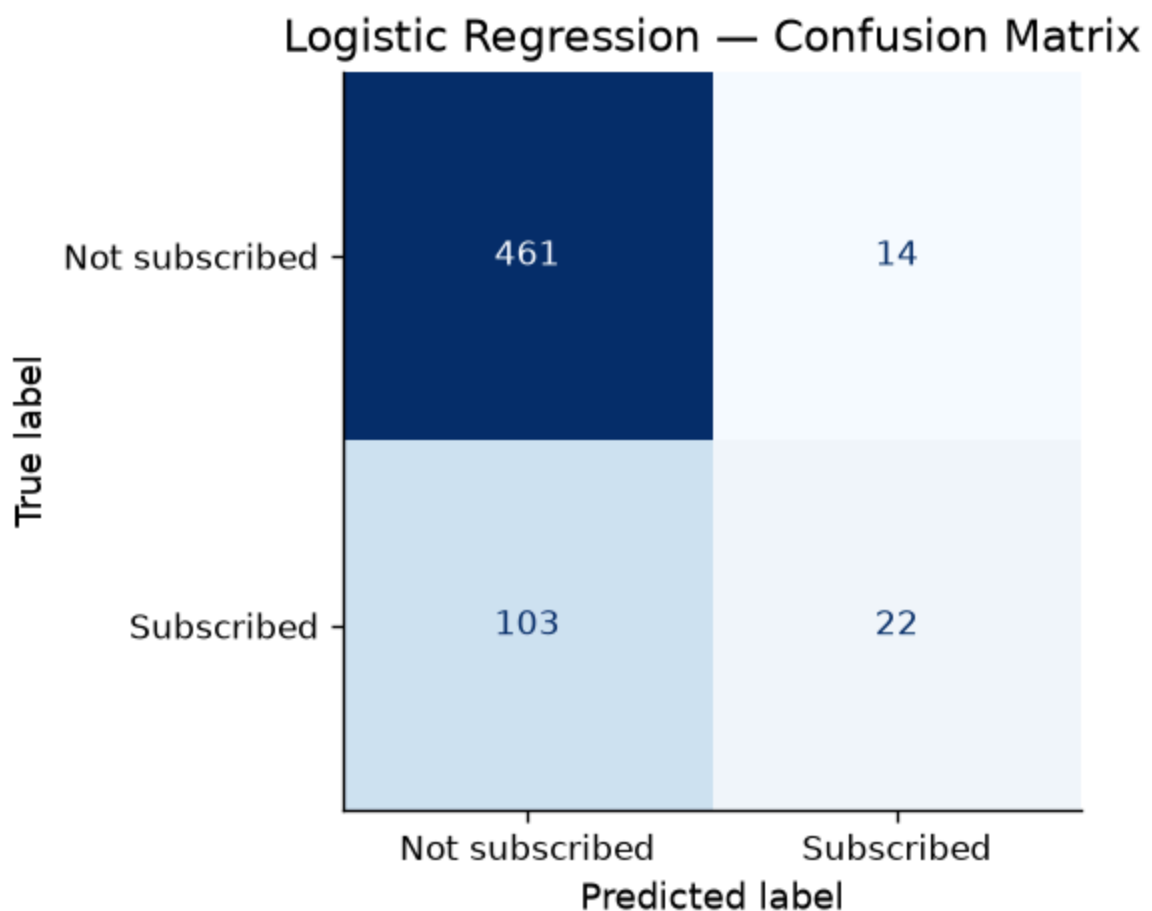
```
Train: (2400, 17) | Test: (600, 17)
Train positive rate: 0.208 | Test positive rate: 0.208
```

```

In [25]: # YOUR CODE HERE
# Fit sklearn LogisticRegression
clf_lr = LogisticRegression(max_iter=1000, random_state=42)
clf_lr.fit(X_train, y_train)
y_pred_lr = clf_lr.predict(X_test)
y_prob_lr = clf_lr.predict_proba(X_test)[: , 1]

In [26]: # YOUR CODE HERE
# Confusion matrix heatmap
cm_lr = confusion_matrix(y_test, y_pred_lr)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_lr, display_labels=['Not subscrib
fig, ax = plt.subplots(figsize=(5, 4))
disp.plot(ax=ax, cmap='Blues', colorbar=False)
plt.title('Logistic Regression - Confusion Matrix')
plt.show()

```



```

In [27]: # YOUR CODE HERE
print(classification_report(y_test, y_pred_lr, target_names=['Not s
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob_lr):.4f}")
# Classification report

```

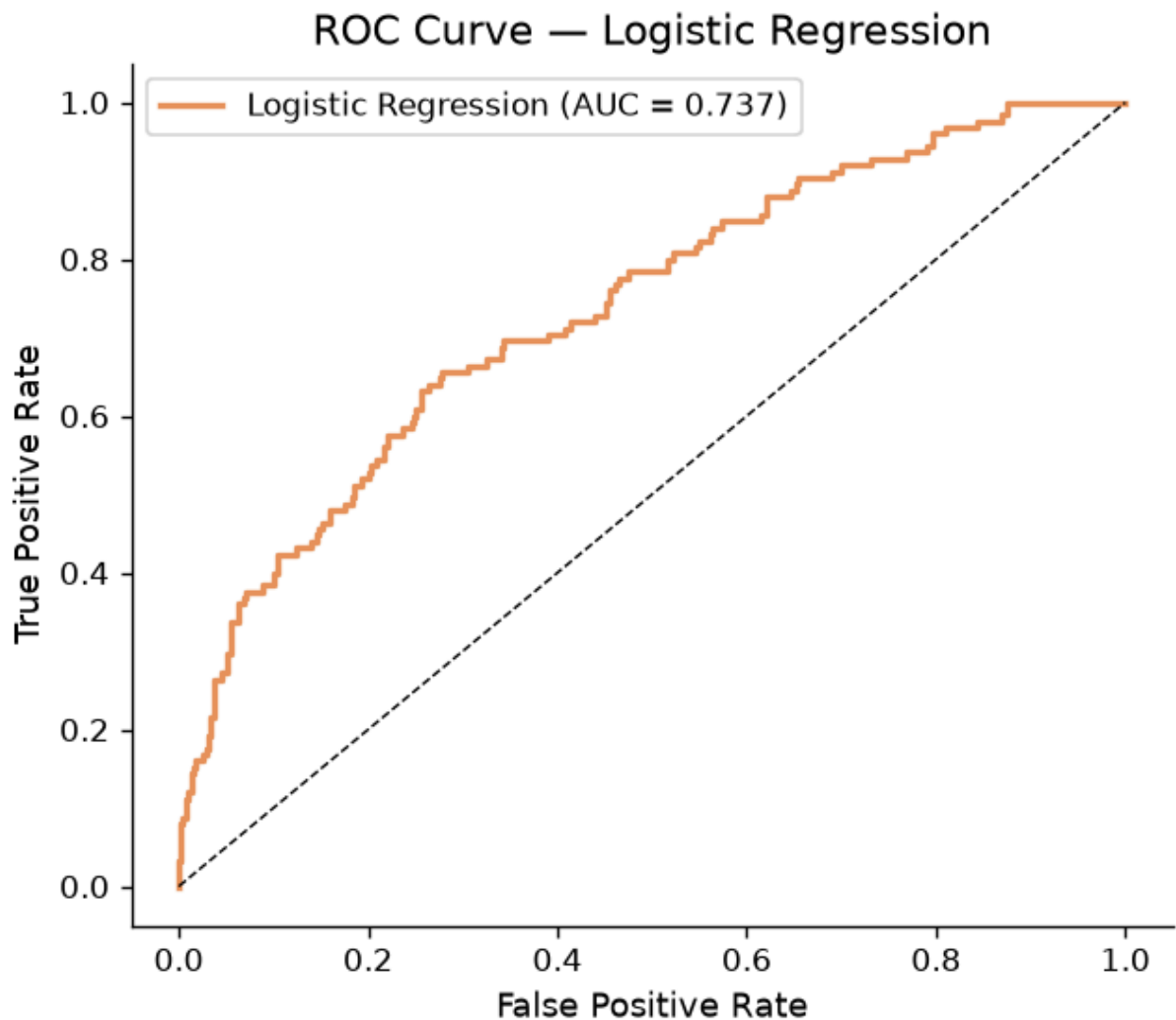
ROC-AUC: 0.7374

```

In [28]: # YOUR CODE HERE
# ROC curve (save fpr_lr, tpr_lr for later comparison with Part C)
fpr_lr, tpr_lr, _ = roc_curve(y_test, y_prob_lr)
auc_lr = roc_auc_score(y_test, y_prob_lr)

```

```
fig, ax = plt.subplots(figsize=(6, 5))
ax.plot(fpr_lr, tpr_lr, color=ACCENT, lw=2, label=f'Logistic Regression (AUC = {auc})')
ax.plot([0,1],[0,1], 'k--', lw=0.8)
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC Curve — Logistic Regression')
ax.legend()
plt.show()
```



Your interpretation (B4):

Which metric did you decide is most important given the class imbalance, and why? Interpret the confusion matrix: how many false negatives does the model produce? From Phigma's business perspective, what is the cost of a false negative (a user who would have subscribed but the model missed)?

Given the class imbalance, accuracy is misleading a model that always predicts "not subscribed" would score 79% without learning anything. ROC-AUC (0.737) is the better metric here since it evaluates performance across all thresholds.

The confusion matrix shows 103 false negatives aka. users who would have subscribed but the model missed. With only 22 true positives, the model catches just 18% of actual subscribers.

From Phigma's perspective, each false negative is a possible missed subscriber. A model with higher recall would be more valuable for a conversion campaign, even if it means flagging some users who would not have subscribed anyway.

Part C — Your Model: Choose, Learn, Apply, Compare

Dataset: `phigma_subscription.csv`

Same train/test split as B4. Do not re-split.

C1 — Choose Your Model

Available options: Decision Tree, Random Forest, Gradient Boosting, XGBoost, LightGBM, K-Nearest Neighbors, Naive Bayes, SVM, LDA, AdaBoost.

 **Your chosen model:** `Random Forest`

Conceptual explanation (write this before any code):

3–5 sentences. How does this model learn from training data? What does it optimize? How is it structurally different from logistic regression? Why might it perform differently on this particular dataset given what you know about the features?

A Random Forest builds many decision trees on random subsets of the data and features, then takes a majority vote across all trees. It optimizes for reducing impurity at each split rather than maximizing a likelihood function. Unlike logistic regression, it makes no assumption of linearity and can capture non-linear effects and feature interactions automatically. `used_premium_feature_preview` is binary with a large effect (39% vs 14% conversion), a decision tree captures this with a single yes/no split, while logistic regression must encode it as a coefficient in a linear equation. The correlated features (`num_sessions_30d` and `total_time_min_30d`) are also less problematic since each tree only uses a random subset of features at each split.

C2 — Fit and Evaluate

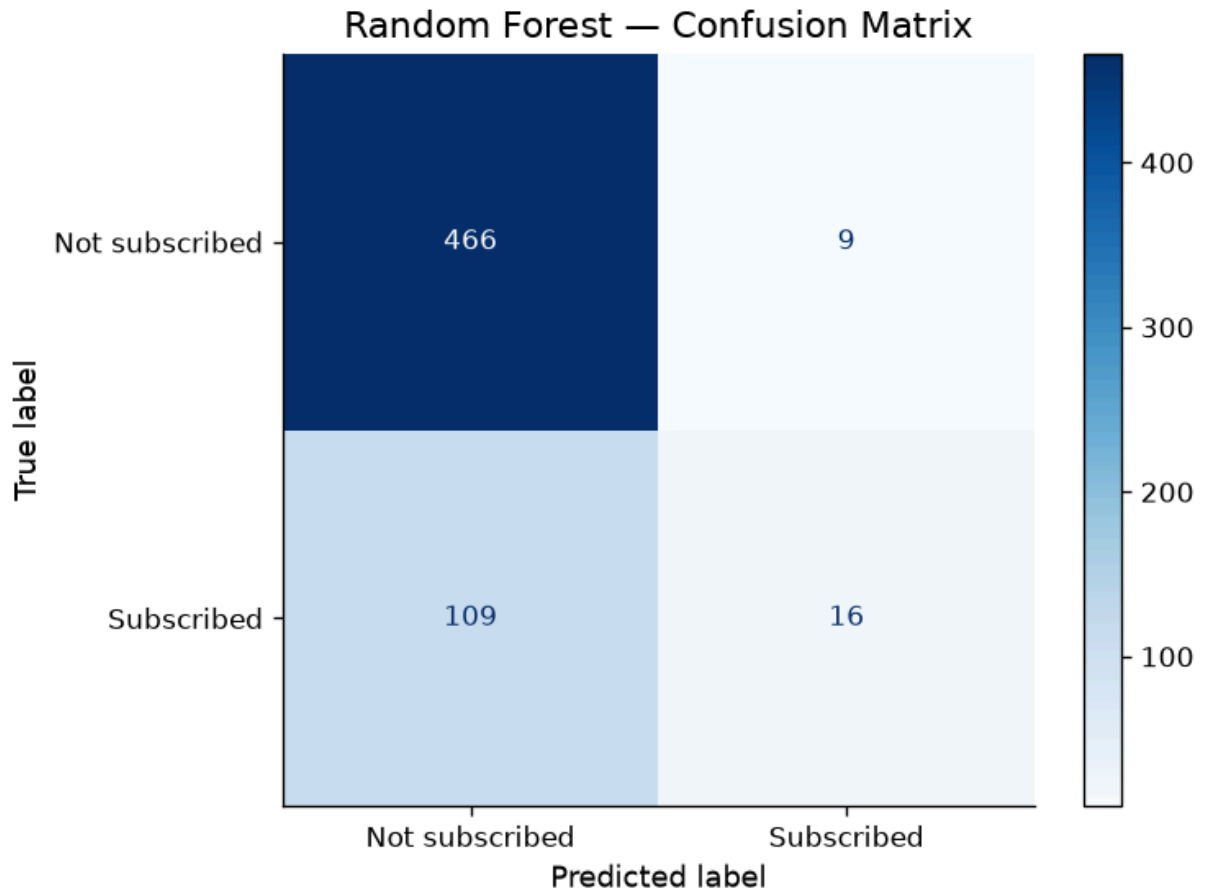
```
In [29]: # YOUR CODE HERE
# Import your chosen model, e.g.:
from sklearn.ensemble import RandomForestClassifier
# from sklearn.tree import DecisionTreeClassifier
```

```
# from sklearn.neighbors import KNeighborsClassifier
# from sklearn.naive_bayes import GaussianNB
# from sklearn.svm import SVC
# from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
# from sklearn.ensemble import GradientBoostingClassifier, AdaBoostClassifier
# from xgboost import XGBClassifier # pip install xgboost
# from lightgbm import LGBMClassifier # pip install lightgbm

# Fit with default hyperparameters
clf_c = RandomForestClassifier(random_state=42) # add random_state if supported
clf_c.fit(X_train, y_train)
y_pred_c = clf_c.predict(X_test)
y_prob_c = clf_c.predict_proba(X_test)[: , 1] # omit if model has no predict_proba
```

```
In [30]: # YOUR CODE HERE
# Confusion matrix

cm = confusion_matrix(y_test, y_pred_c)
disp = ConfusionMatrixDisplay(cm, display_labels=['Not subscribed', 'Subscribed'])
disp.plot(cmap='Blues')
plt.title('Random Forest - Confusion Matrix')
plt.tight_layout()
plt.show()
```



```
In [31]: # YOUR CODE HERE
# Classification report + ROC-AUC
```

```
print(classification_report(y_test, y_pred_c, target_names=['Not subscribed', 'Subs
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob_c):.4f}")
```

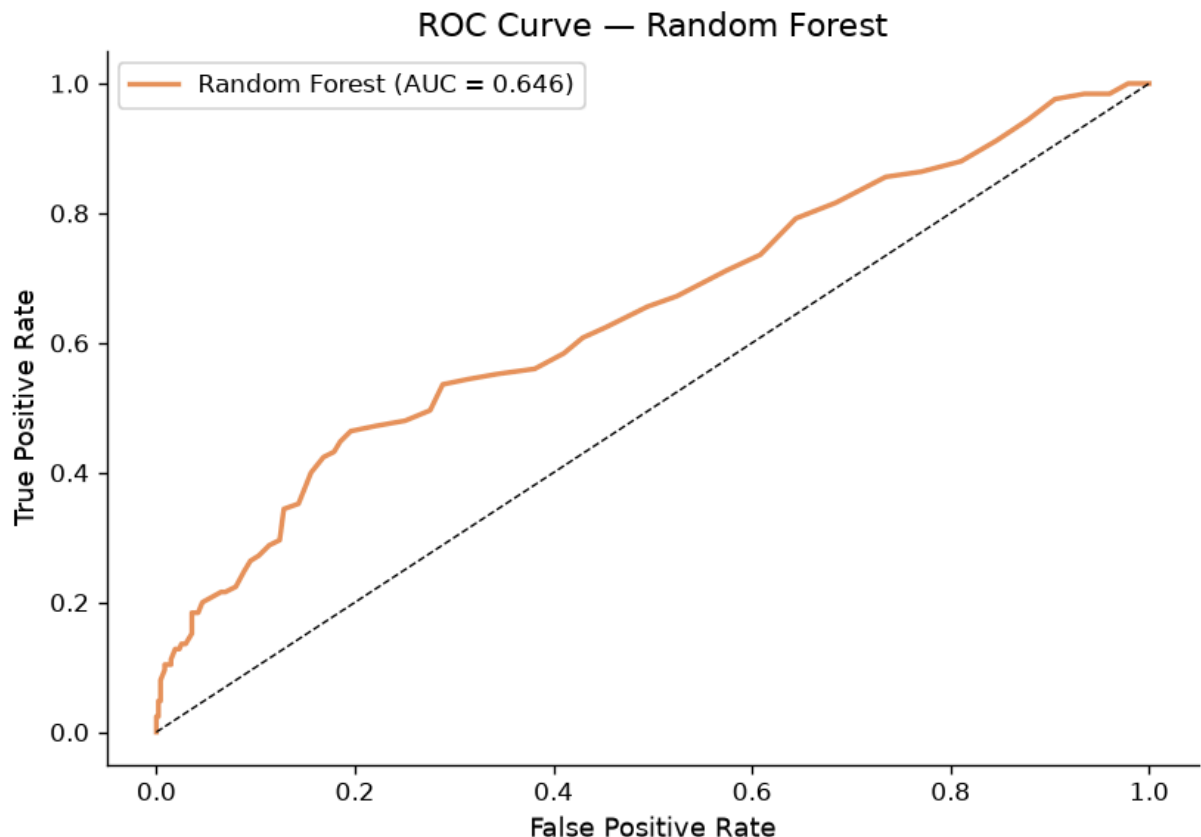
	precision	recall	f1-score	support
Not subscribed	0.81	0.98	0.89	475
Subscribed	0.64	0.13	0.21	125
accuracy			0.80	600
macro avg	0.73	0.55	0.55	600
weighted avg	0.77	0.80	0.75	600

ROC-AUC: 0.6460

```
In [32]: # YOUR CODE HERE
# ROC curve (save fpr_c, tpr_c, auc_c for comparison plot)

fpr_c, tpr_c, _ = roc_curve(y_test, y_prob_c)
auc_c = roc_auc_score(y_test, y_prob_c)

plt.figure(figsize=(7, 5))
plt.plot(fpr_c, tpr_c, color=ACCENT, lw=2, label=f'Random Forest (AUC = {auc_c:.3f})
plt.plot([0, 1], [0, 1], 'k--', lw=0.8)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve — Random Forest')
plt.legend()
plt.tight_layout()
plt.show()
```



C3 — Model Comparison

```
In [33]: # YOUR CODE HERE
# Summary comparison table

def class1_metrics(y_true, y_pred, y_prob):
    report = classification_report(y_true, y_pred, output_dict=True)
    return {
        'ROC-AUC': roc_auc_score(y_true, y_prob),
        'F1 (subscribed)': report['1']['f1-score'],
        'Precision (subscribed)': report['1']['precision'],
        'Recall (subscribed)': report['1']['recall'],
    }

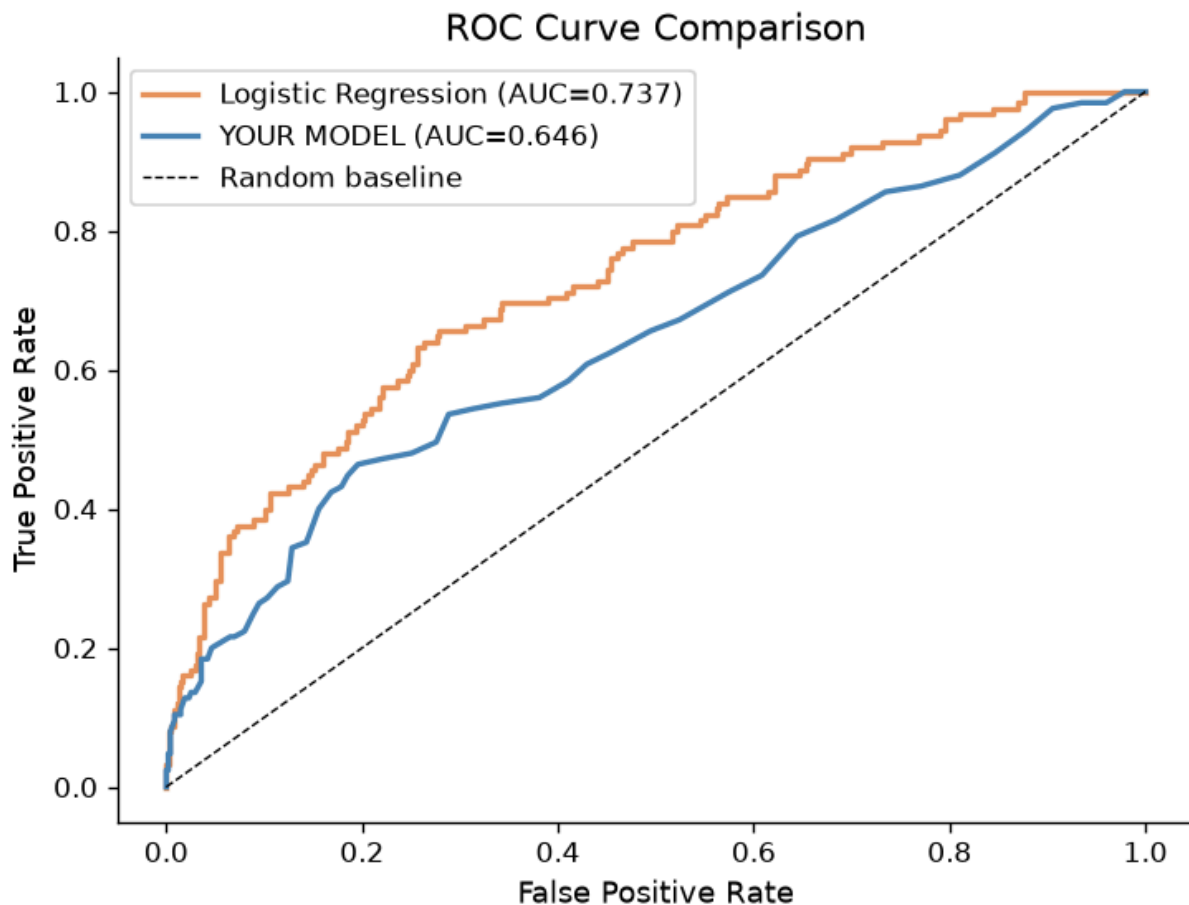
metrics = {
    'Logistic Regression': class1_metrics(y_test, y_pred_lr, y_prob_lr),
    'Random Forest': class1_metrics(y_test, y_pred_c, y_prob_c),
}

pd.DataFrame(metrics).T.round(4)
```

```
Out[33]:
```

	ROC-AUC	F1 (subscribed)	Precision (subscribed)	Recall (subscribed)
Logistic Regression	0.7374	0.2733	0.6111	0.176
Random Forest	0.6460	0.2133	0.6400	0.128

```
In [34]: # YOUR CODE HERE
# Dual ROC curve plot
fig, ax = plt.subplots(figsize=(7, 5))
ax.plot(fpr_lr, tpr_lr, color=ACCENT, lw=2, label=f'Logistic Regression (AUC={auc_l})')
ax.plot(fpr_c, tpr_c, color='steelblue', lw=2, label=f'YOUR MODEL (AUC={auc_c:.3f})')
ax.plot([0,1],[0,1], 'k--', lw=0.8, label='Random baseline')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC Curve Comparison')
ax.legend()
plt.show()
```



Comparison paragraph (C3):

*Which model performs better, and by how much? Explain *why* the performance difference occurred — based on what you know about how each model works and the structure of this dataset (non-linearities, class imbalance, number of features). If your model performed worse than logistic regression, that is a valid finding — explain why.**

Logistic regression outperformed Random Forest across all metrics (ROC-AUC 0.737 vs 0.646, F1 0.273 vs 0.213, Recall 0.176 vs 0.128). This could be because the dataset's relationships are largely linear. The odds ratio analysis showed that most predictors have modest, additive effects on subscription probability, which logistic regression models directly. Random Forest also needs more data to reliably learn treesplits, and with only ~500 positive examples the ensemble may be overfitting to noise rather than signal. The `class_weight='balanced'` setting also did not improve recall meaningfully, suggesting the imbalance is not the main issue.

C4 — Bonus: Hyperparameter Tuning (+5 points)

Complete this section only if you want to attempt the bonus.

```
In [35]: from sklearn.model_selection import GridSearchCV
```



```

# Define a parameter grid relevant to your chosen model
param_grid = {
    'n_estimators': [100, 300],
    'max_depth': [None, 10, 20],
    'min_samples_leaf': [1, 4],
}

# Run cross-validated search on the TRAINING SET ONLY
search = GridSearchCV(
    RandomForestClassifier(class_weight='balanced', random_state=42),
    param_grid, cv=5, scoring='roc_auc', n_jobs=-1
)
search.fit(X_train, y_train)

print(f"Best parameters: {search.best_params_}")
print(f"Best CV AUC: {search.best_score_:.4f}")

# Evaluate tuned model on test set
y_pred_tuned = search.best_estimator_.predict(X_test)
y_prob_tuned = search.best_estimator_.predict_proba(X_test)[:, 1]
print(f"Tuned test AUC: {roc_auc_score(y_test, y_prob_tuned):.4f}")
print(f"Default test AUC: {auc_c:.4f}")

```

Best parameters: {'max_depth': 10, 'min_samples_leaf': 4, 'n_estimators': 300}
 Best CV AUC: 0.7233
 Tuned test AUC: 0.7044
 Default test AUC: 0.6460

Submission Checklist

Before submitting, confirm:

- ☐ All `# YOUR CODE HERE` cells are completed and run without errors
- ☐ All `**  Your interpretation**` cells contain your own written analysis
- ☐ The notebook runs top-to-bottom without errors (`Kernel > Restart & Run All`)
- ☐ File is named `iat461_a2_[YourName]_[SFUid].ipynb`
- ☐ PDF export is named `iat461_a2_[YourName]_[SFUid].pdf`
- ☐ All AI-assisted code blocks include `#BEGIN` / `#END` disclosure comments